DATA SCIENCE INTERNSHIP

TASK 2

Titanic Survival Prediction Using Machine Learning

Project Type: Classification
Language: Python
Libraries: pandas, NumPy, Matplotlib, scikit-learn

Beginner-friendly project document containing the objective, complete code, explanation, sample output, step-by-step instructions, and report.

1. Project Overview

Objective: Build a machine-learning model that predicts whether a Titanic passenger survived based on information such as passenger class, sex, age, family aboard, ticket fare, and port of embarkation.

Problem type: Binary classification. The target column is `Survived`: 0 means did not survive and 1 means survived.

2. Tools and Libraries

• Python — programming language

• pandas — loading and preparing tabular data

• NumPy — numerical operations

• Matplotlib — visualizing results

• scikit-learn — preprocessing, model training, and evaluation

3. Step-by-Step Instructions

1. Install the required libraries: `pip install pandas numpy matplotlib scikit-learn`.

2. Download a Titanic training CSV that includes the columns used below, including the target column `Survived`. Save it as `train.csv` in the same folder as the Python script.

3. Run the code in Jupyter Notebook, Google Colab, or a Python IDE. In Colab, upload `train.csv` when prompted.

4. The script cleans missing values, converts categorical columns, trains Logistic Regression, and prints evaluation results.

4. Complete Python Code

```
# Task 2: Titanic Survival Prediction
# Dataset: train.csv (Kaggle Titanic training dataset)

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    ConfusionMatrixDisplay
)

# 1. Load the dataset
if not os.path.exists("train.csv"):
    try:
        from google.colab import files
        uploaded = files.upload()
        if "train.csv" not in uploaded:
            raise FileNotFoundError("Please upload a file named train.csv.")
    except ImportError:
        raise FileNotFoundError(
            "Place the Titanic training dataset in this folder as train.csv."
        )

df = pd.read_csv("train.csv")
print("Dataset shape:", df.shape)
print(df.head())

# 2. Select useful input features and target
features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
target = "Survived"

missing_columns = [c for c in features + [target] if c not in df.columns]
```

```
if missing_columns:
    raise ValueError(f"Dataset is missing columns: {missing_columns}")

X = df[features]
y = df[target].astype(int)

# 3. Separate numeric and categorical columns
numeric_features = ["Pclass", "Age", "SibSp", "Parch", "Fare"]
categorical_features = ["Sex", "Embarked"]

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])

# 4. Create a machine-learning pipeline
model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

# 5. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 6. Train the model
model.fit(X_train, y_train)

# 7. Make predictions and evaluate
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\\nAccuracy:", round(accuracy * 100, 2), "%")
print("\\nClassification report:\\n")
print(classification_report(y_test, y_pred, zero_division=0))

cm = confusion_matrix(y_test, y_pred)
print("Confusion matrix:\\n", cm)

ConfusionMatrixDisplay(confusion_matrix=cm).plot()
plt.title("Titanic Survival Prediction - Confusion Matrix")
plt.tight_layout()
plt.savefig("titanic_confusion_matrix.png", dpi=150)
plt.show()

# 8. Example prediction for a hypothetical passenger
sample_passenger = pd.DataFrame([{
    "Pclass": 3,
    "Sex": "female",
    "Age": 25,
    "SibSp": 0,
    "Parch": 0,
    "Fare": 10.0,
    "Embarked": "S"
}])

prediction = model.predict(sample_passenger)[0]
probability = model.predict_proba(sample_passenger)[0][1]

print("\\nExample passenger prediction:")
print("Predicted class:", "Survived" if prediction == 1 else "Did not survive")
print("Estimated survival probability:", round(probability * 100, 2), "%")

# Save predictions for the test set
results = X_test.copy()
results["Actual_Survived"] = y_test
results["Predicted_Survived"] = y_pred
results.to_csv("titanic_predictions.csv", index=False)
print("\\nSaved: titanic_predictions.csv and titanic_confusion_matrix.png")
```

5. Code Explanation

Load data: pandas reads train.csv into a DataFrame. The dataset must contain the selected feature columns and Survived.

Choose features: Pclass, Sex, Age, SibSp, Parch, Fare, and Embarked are inputs. Survived is the answer the model learns to predict.

Handle missing values: Numeric missing values are filled with each column's median. Categorical missing values are filled with the most common category.

Encode categories: OneHotEncoder converts text categories such as Sex and Embarked into numeric columns that the model can use.

Scale numeric data: StandardScaler puts numeric features on comparable scales.

Split data: 80% of rows train the model and 20% test it. random_state=42 makes the split repeatable.

Train and evaluate: Logistic Regression learns patterns from the training data. Accuracy and the classification report summarize performance; the confusion matrix shows correct and incorrect predictions.

Predict a sample: The sample passenger is hypothetical. The probability is the model's estimate, not a guarantee.

6. Sample Output

Exact numbers can vary depending on the dataset version and scikit-learn version. A typical run prints a dataset shape and first rows, followed by output similar to this:

```
Dataset shape: (891, 12)

Accuracy: (your result) %

Classification report:
              precision    recall  f1-score   support
           0       ...
           1       ...

Confusion matrix:
 [[... ...]
  [... ...]]

Example passenger prediction:
Predicted class: Survived  (or Did not survive)
Estimated survival probability: (model estimate) %

Saved: titanic_predictions.csv and titanic_confusion_matrix.png
```

Important: the sample output above is a format example, not a claimed model result. Run the code with your own train.csv to obtain actual values.

7. Expected Files

• titanic_predictions.csv — test-set rows with actual and predicted survival labels.

• titanic_confusion_matrix.png — saved confusion-matrix chart.

8. Project Report

Title: Titanic Survival Prediction Using Machine Learning

Introduction: This project applies supervised machine learning to the Titanic passenger dataset. The goal is to estimate whether a passenger survived using selected passenger details.

Methodology: The dataset is loaded and checked for required columns. Relevant features are selected. Missing values are imputed, categorical features are one-hot encoded, numeric values are scaled, and Logistic Regression is trained on an 80% training subset. The remaining 20% is used for testing.

Evaluation: Accuracy, precision, recall, F1-score, and a confusion matrix are printed. Actual scores depend on the dataset and environment; report the values from your own run.

Conclusion: The project demonstrates a complete classification workflow, including preprocessing, model training, evaluation, and prediction. It is an educational exercise; predictions are not historical certainty and should not be

treated as causal explanations.

9. Troubleshooting

• FileNotFoundError: confirm the CSV is named train.csv and is in the working folder, or upload it in Colab.

• Missing columns: use the standard Titanic training CSV with Survived, Pclass, Sex, Age, SibSp, Parch, Fare, and Embarked.

• Import error: install libraries using pip install pandas numpy matplotlib scikit-learn.

• No graph window: check that titanic_confusion_matrix.png was saved in the working folder.